

Object-oriented Programming

Week 12

Abstract Class

- There are cases in which it's useful to define classes that are never instantiated. Such classes are called **abstract base classes** or simply **abstract classes**
- Abstract base classes are incomplete — derived classes must define the “missing pieces”

Abstract Class

- An abstract class provides a base class from which other classes can inherit
- Classes that can be used to instantiate objects are called **concrete classes**
- An abstract can inherit another abstract class

Example

- Suppose that we have an application that works with **2D** shapes
- We have a class called **2D** with a function **draw()**. But... what shape would the function draw? Should **draw()** be implemented in base class??
- We can extend the class with derived classes such as **Square**, **Triangle** & **Circle**. And provide implementation for **draw()**

Abstract Class

- Abstract base classes are too generic to define real objects
- Concrete classes provide the specifics that make it reasonable to instantiate objects

Pure Virtual Functions

- A class is made abstract by declaring one or more of its virtual functions to be “**pure**”
- A pure virtual function is specified by placing “**= 0**” in its declaration
- Example: **virtual void draw() = 0;**

Virtual vs Pure Virtual

- A virtual function has an implementation in the base class; a pure virtual function does not have an implementation in the base class
- Virtual functions *can be* overridden by the derived classes; pure virtual functions *must be* overridden by the derived classes

A class is abstract if it has at least one pure virtual function.

- `class Base{`
- `int x;`
- `public:`
- `virtual void fun() = 0;`
- `int getX() {`
- `return x; };`
- `// This class inherits from Base and implements fun()`
- `class Derived: public Base{`
- `int y;`
- `public:`
- `void fun() { cout << "fun() called"; };`
- `int main(void){`
- `Derived d;`
- `d.fun();`
- `return 0;}`

 C:\Users\Nida Munawar\Documents\Untitled1.exe

```
fun() called
```

```
-----
```

```
Process exited after 0.5682 seconds with return value 0
```

```
Press any key to continue . . .
```

We can not instantiate an abstract class

- `#include<iostream>`
- `using namespace std;`
- `class Test{`
- `int x;`
- `public:`
- `virtual void show() = 0;`
- `int getX() {`
- `return x; };`
- `int main(void){`
- `Test t;`
- `return 0;}`

In function 'int main()':

[Error] cannot declare variable 't' to be of abstract type 'Test'

[Note] because the following virtual functions are pure within 'Test':

[Note] virtual void Test::show()

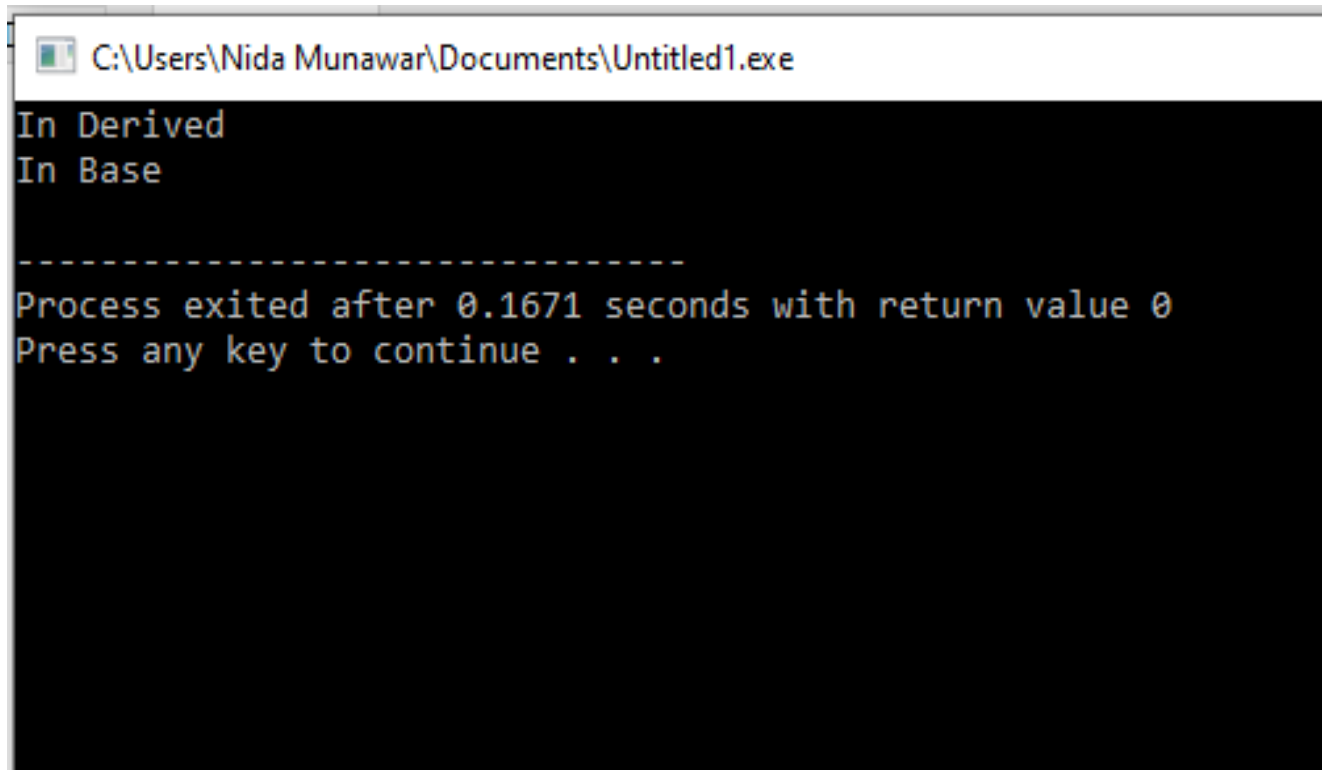
Abstract Class

- Although we cannot instantiate objects of an abstract base class, we *can use the* abstract base class to declare pointers and references that can refer to objects of its derived class
- Abstract classes support the principle of Polymorphism

We can have pointers and references of abstract class type.

- class Base{
- int x;
- public:
- void getX() { cout << "In Base \n"; }
- virtual void show() = 0;};
- class Derived: public Base{
- public:
- void show() { cout << "In Derived \n"; };
- int main(void){
- Base *bp = new Derived();
- bp->show();
- bp->getX();
- return 0;}

We can have pointers and references of abstract class type.



```
C:\Users\Nida Munawar\Documents\Untitled1.exe  
In Derived  
In Base  
  
-----  
Process exited after 0.1671 seconds with return value 0  
Press any key to continue . . .
```

- *If we do not override the pure virtual function in derived class, then derived class also becomes abstract class.*
- **class Base{**
- **public:**
- **virtual void show() = 0;;**
- **class Derived : public Base { };**
- **int main(void){**
- **Derived d;**
- **return 0;}**

In function 'int main()':

[Error] cannot declare variable 'd' to be of abstract type 'Derived'

[Note] because the following virtual functions are pure within 'Derived':

[Note] virtual void Base::show()

- class Base{
- public:
- virtual void show() = 0;};
- class Derived : public Base { };
- class Derived1 : public Derived {
- public:
- void show() {
- cout<<"show method implementation \n";} };
- int main(void){
- Derived1 d;
- d.show();
- return 0;}

 C:\Users\Nida Munawar\Documents\Untitled1.exe

```
show method implementation
```

```
-----
```

```
Process exited after 0.1622 seconds with return value 0
```

```
Press any key to continue . . .
```

An abstract class can have constructors.

- class Base{
- protected:
- int x;
- public:
- virtual void fun() = 0;
- Base(int i) {
- x = i;
- cout<<"Constructor of base called\n"; };
- class Derived: public Base{
- int y;
- public:
- Derived(int i, int j):Base(i) { y = j; }
- void fun() { cout << "x = " << x << ", y = " << y<<'\n'; };
-

An abstract class can have constructors.

- `int main(void){`
- `Derived d(4, 5);`
- `d.fun();`
- `//object creation using pointer of base class`
- `Base *ptr=new Derived(6,7);`
- `ptr->fun();`
- `return 0;}`

An abstract class can have constructors.

 C:\Users\Nida Munawar\Documents\Untitled1.exe

```
Constructor of base called
```

```
x = 4, y = 5
```

```
Constructor of base called
```

```
x = 6, y = 7
```

```
-----
```

```
Process exited after 0.2104 seconds with return value 0
```

```
Press any key to continue . . .
```

Case Study

A company pays its employees weekly.

The employees are of four types: **Salaried employees** are paid a fixed weekly salary regardless of the number of hours worked, **hourly employees** are paid by the hour and receive overtime pay for all hours worked in excess of 40 hours, **commission employees** are paid a percentage of their sales and **base-salary-plus-commission employees** receive a base salary plus a % of their sales. For the current pay period, the company has decided to reward base-salary-plus-commission employees by adding 10 percent to their base salaries.

Next Lecture

- Revision